Study book

Topic 9: JOINs, GROUP BY, subqueries, transactions

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 2, topic 9 of 20.

Brief overview

What this lesson is about. Questions that span **two tables** (JOIN), questions that **summarize** (GROUP BY, HAVING, aggregates), questions about **existence** (subqueries, EXISTS), and changes that must happen **all or nothing** (transactions, locks).
What we built. `sql/06`: a schema change (`DueDate`) on a table with data, what an EF migration does. `sql/07–10`: JOINs and **Report 1: overdue tasks**, GROUP BY and **Report 2: tasks per project**, subqueries and EXISTS, transactions. Plus a real **blocking experiment** with two sessions.
Checkpoint: explain **why a JOIN is needed** and **what a transaction protects**. Passed after one re-check round.
Four traps run through the whole topic, and three of them are Topic 8's NULL rule in disguise: the **LEFT JOIN + WHERE** trap, the **COUNT(*)** trap, the **NOT IN** trap, and SQL Server's "**an error doesn't undo the transaction**" trap.
Why it matters for C#: in Topics 10–11, EF Core generates all of this. `Include` and navigation properties become JOINs, `GroupBy` becomes GROUP BY, `Any()` becomes EXISTS, and `SaveChanges` runs in a transaction.

Contents

Part	What is inside
1	Why a JOIN is needed: normalization, the N+1 alternative (the checkpoint)
2	INNER vs LEFT JOIN, aliases, the ON vs WHERE trap
3	The full logical order of a query
4	GROUP BY, aggregates, COUNT(*) vs COUNT(col), HAVING, conditional aggregation
5	Subqueries, IN, EXISTS, the NOT IN trap
6	Transactions: ACID, autocommit, the XACT_ABORT trap, the correct pattern, EF Core (the checkpoint)
7	Locks and isolation: the real blocking experiment
8	Our scripts (06–10) and their real output
9	DEBUG tasks with answers
10	Your answers, corrections, interview Q&A
11	Summary, cheat sheet, checklist, what comes next

Part 1. Why a JOIN is needed

In Topic 8 we split the data on purpose: a project's name is stored **once**, in `Projects`. A task stores only `ProjectId`. This is **normalization**: each fact lives in one place, so renaming a project is **one** UPDATE, not one per task, and two copies can never disagree.

The price: "show tasks **with their project name**" needs **both** tables. A JOIN puts the rows back together through the foreign key:

```
SELECT t.Id, t.Title, p.Name AS ProjectName
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId;
```

In TypeScript terms, it's the lookup you'd write in the frontend, but done **inside the database, in one query**:

```
const projectsById = new Map(projects.map(p => [p.id, p]));
const rows = tasks.map(t => ({ ...t, projectName: projectsById.get(t.projectId)!.name }));
```

1.1 The alternatives without a JOIN, and why they're bad

Alternative	What happens	Problem
N+1 queries	1 query for the tasks, then 1 query **per task** for its project	7 tasks = 8 round trips; 1,000 tasks = 1,001. Each costs network time. The classic ORM performance bug (Topic 11)
Duplicated data	copy `ProjectName` into every task row	a rename must update every copy; miss one and the data disagrees with itself
JOIN	one query; the database matches FK = PK (using `IX_Tasks_ProjectId` or a hash match)	none: this is what databases are built for

Your first answer, corrected

Your first answer said that without a JOIN "the DB would need to scan one table, then the second, and match them". That is actually what a JOIN **does inside** the database (one of its strategies). Without a JOIN, the extra work moves into **your code**: N+1 round trips. Your re-check answer got it right: *"without it WE would need to query one table, and then query per each row"*.

Checkpoint answer, part 1

Why is a JOIN needed? The data is split across tables so each fact is stored once (normalization). A JOIN combines the rows again **in one query** by matching the FK to the PK. Without it you'd need N+1 queries from the application, or duplicated data.

- **Aliases** (`AS t`, `AS p`) are short names for the tables. With two tables, always write `t.Title`: both tables have `Id` and `CreatedAt`, so a bare `Id` is an **ambiguous column** error.
- `ON` says how rows match: almost always **FK = PK**.

Part 2. INNER JOIN vs LEFT JOIN

	Keeps	Our data
INNER JOIN	only rows with a match **on both sides**	7 rows; "Internal tools" (no tasks) is **missing**

	Keeps	Our data
LEFT JOIN	**every row of the left table** (the one after FROM), matched where possible, otherwise **NULL** in the right table's columns	8 rows: "Internal tools" once, with `TaskId` and `Title` NULL

```
Projects LEFT JOIN Tasks:

 Website redesign -- Design mockups
 Website redesign -- Build landing page        INNER JOIN keeps only rows with a match
 Website redesign -- SEO audit                 on both sides: these 7.
 Website redesign -- Write copy
 Mobile app       -- Login screen
 Mobile app       -- Push notifications
 Mobile app       -- App store listing
 Internal tools   -- NULL                       <- only LEFT JOIN keeps this one
```

- **How to choose:** "should a project with **zero** tasks still appear?" Yes (reports, dashboards): LEFT. No (tasks with their project name): INNER.
- `RIGHT JOIN` = LEFT JOIN with the tables swapped (write it as LEFT). `FULL JOIN` keeps unmatched rows of both sides. `CROSS JOIN` pairs every row with every row. Rarely needed.
- **LINQ:** INNER = `Join(...)`. LEFT = `GroupJoin(...).SelectMany(..., DefaultIfEmpty())` (awkward). With EF Core you write `t.Project.Name` and EF writes the JOIN (Topic 11).

Your answer, sharpened

You answered "inner join when I need only **truthy** rows, left join **all** rows". Two fixes: INNER keeps rows with a **match on both sides** (nothing to do with truthiness), and LEFT keeps all rows **of the left table**, not of both tables.

2.1 The LEFT JOIN trap: ON vs WHERE

"Every project, with its **open** tasks":

```
-- J3a, WRONG: "Internal tools" disappears
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
WHERE t.Status <> 2

-- J3b, RIGHT: the condition decides WHICH TASKS match; every project stays
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id AND t.Status <> 2
```

Why: WHERE runs **after** the join. The "Internal tools" row has `t.Status = NULL`, and `NULL <> 2` is **UNKNOWN** (Topic 8's three-valued logic). WHERE keeps only TRUE rows, so the row is thrown away, and the LEFT JOIN silently became an INNER JOIN. Real result: J3a 5 rows, J3b 6 rows.

Rule

In a LEFT JOIN, conditions on the **right** table go in **ON**. Conditions in WHERE on the right table undo the LEFT.

Part 3. The full logical order of a query

```
FROM / JOIN  ->  WHERE  ->  GROUP BY  ->  HAVING  ->  SELECT  ->  ORDER BY  ->  OFFSET / FETCH
 which rows      filter     make         filter      compute     sort          page
 (combined)      rows       groups       groups      columns
```

Most rules of this topic follow from this one picture:

- `WHERE COUNT(*) > 2` is an **error**: when WHERE runs, nothing has been counted yet. Use HAVING.
- A SELECT alias (`AS DaysOverdue`) can't be used in WHERE (SELECT runs later), but it can be used in ORDER BY.
- The LEFT JOIN trap: WHERE runs **after** the JOIN.
- LINQ's method chain (`Where` before `GroupBy` before `Select`) follows the same order. That's why it feels natural once you know SQL's real order.

Part 4. GROUP BY and aggregates

Aggregates turn many rows into one value: `COUNT`, `SUM`, `MIN`, `MAX`, `AVG`. **GROUP BY** = "one output row **per distinct value**", like JS `Object.groupBy(tasks, t => t.status)` followed by a count per group.

SQL	LINQ
`SELECT COUNT(*) FROM dbo.Tasks`	`tasks.Count()`
`SELECT MAX(CreatedAt) FROM dbo.Tasks`	`tasks.Max(t => t.CreatedAt)`
`SELECT Status, COUNT(*) FROM dbo.Tasks GROUP BY Status`	`tasks.GroupBy(t => t.Status).Select(g => new { g.Key, Count = g.Count() })`
`... WHERE ... GROUP BY ... HAVING COUNT(*) > 2`	`.Where(...).GroupBy(...).Where(g => g.Count() > 2)`

The rule: every column in SELECT is either **in GROUP BY** or **inside an aggregate**. `SELECT Status, Title, COUNT(*) ... GROUP BY Status` is an error: a group of 3 Todo tasks has 3 different titles, and the database won't guess which one to show.

4.1 COUNT(*) vs COUNT(column): the second trap

- `COUNT(*)` counts **rows**.
- `COUNT(column)` counts **non-NULL values** in that column. G1: `COUNT(*)` = 7, `COUNT(DueDate)` = **6** (task 6 has no due date).

After a LEFT JOIN they differ exactly where it hurts. "Internal tools" has **one row** (filled with NULLs), so `COUNT(*)` says **1 task**:

Real output of G3: COUNT(*) vs COUNT(t.Id)

```
Name             Tasks_WRONG Tasks_RIGHT
Internal tools   1           0
Mobile app       3           3
Website redesign 4           4
```

Rule
After a LEFT JOIN, count a column of the right table (`COUNT(t.Id)`), never `COUNT(*)`.

Your answer, sharpened

You wrote: "`*` counts all rows, even unreal ones (**falsy**); `t.Id` all valid **truthy** rows". The idea is right, but drop the JS words in SQL. `COUNT(col)` skips **only NULL**. A `0` or an empty string `''` **is** counted, although both are falsy in JS. Saying "falsy" in an interview signals JS thinking.

4.2 HAVING = WHERE for groups

```
WHERE t.Status <> 2          -- filters ROWS, before grouping (drop Done tasks)
GROUP BY p.Id, p.Name
HAVING COUNT(*) > 2          -- filters GROUPS, after counting
```

G4 result: only **Mobile app** (3 open tasks). Website redesign has 2 open tasks, so `> 2` drops it.

4.3 Conditional aggregation: several counts in one pass

```
SUM(CASE WHEN t.Status = 0 THEN 1 ELSE 0 END) AS Todo
```

`CASE WHEN ... THEN ... ELSE ... END` is SQL's ternary (`status === 0 ? 1 : 0`). Summing the 1s counts the Todo tasks. One CASE per status gives a full status breakdown per project in **one query**: that's **Report 2**. For "Internal tools" `t.Status` is NULL, so every CASE takes the ELSE (0).

Part 5. Subqueries and EXISTS

A **subquery** is a query inside another query, in parentheses.

Kind	Returns	Example (script 09)	LINQ
scalar	one value	`WHERE CreatedAt = (SELECT MAX(CreatedAt) FROM dbo.Tasks)`	`var max = tasks.Max(...); tasks.Where(t => t.CreatedAt == max)`
IN	a list	`WHERE Id IN (SELECT ProjectId FROM dbo.Tasks WHERE Status = 2)`	`ids.Contains(p.Id)`
EXISTS	true / false	`WHERE EXISTS (SELECT 1 FROM dbo.Tasks AS t WHERE t.ProjectId = p.Id AND ...)`	`tasks.Any(t => t.ProjectId == p.Id && ...)`

EXISTS is SQL's `.some()` / `.Any()`: "does the subquery return **at least one row**?"

- The subquery uses `p.Id` from the **outer** query: a **correlated** subquery, checked once per project.
- `SELECT 1` is a convention: EXISTS ignores what's selected and only checks whether a row exists.
- The database can **stop at the first match**. `COUNT(*) > 0` would count them all, so EXISTS is the better way to ask "is there any?".
- `NOT EXISTS` = "has **no** matching row". `LEFT JOIN ... WHERE t.Id IS NULL` gives the same answer (S4 shows both). You'll see both styles in real code.

5.1 The NOT IN trap (NULL trap number three)

`x NOT IN (a, b, NULL)` means `x <> a AND x <> b AND x <> NULL`. The last part is UNKNOWN, so the whole condition is **never TRUE**, and you get **0 rows** with no error.

S5 asks: "tasks whose due date is **not** one of project 2's due dates". Project 2's due dates are (2026-09-30, **NULL**, 2026-10-10), because task 6 has no due date. Result: **0 rows**. S5b asks the same question with `NOT EXISTS`: tasks 1–4.

Rule
Use `NOT EXISTS` instead of `NOT IN (subquery)`, unless the subquery's column is `NOT NULL`.

One rule, three traps
All three NULL traps of this topic come from one rule from Topic 8: **comparisons with NULL are UNKNOWN, and WHERE keeps only TRUE**.
1. The LEFT JOIN + WHERE trap: `NULL <> 2` drops the unmatched row. **2.** The `<>` trap (Topic 8): `<> N'x'` skips NULL rows. **3.** The NOT IN trap: one NULL in the list makes everything UNKNOWN.
COUNT is different: it's not a comparison. `COUNT(col)` simply **skips** NULLs.

Part 6. Transactions

A transaction groups several statements into one unit: all of them happen, or none do.

Example: "delete project 1" is **two** statements, because the FK (Topic 8, C5) forces you to delete its tasks first:

```
BEGIN TRANSACTION;
    DELETE FROM dbo.Tasks    WHERE ProjectId = 1;
    DELETE FROM dbo.Projects WHERE Id = 1;
COMMIT TRANSACTION;     -- make it permanent    (or ROLLBACK TRANSACTION: undo everything since BEGIN)
```

If the connection drops, the server crashes, or the second statement fails **between** them, the tasks are gone and the project remains: half an operation that nothing will ever fix. **TS analogy:** like building a new state object and calling `setState` once at the end, so the UI never sees a half-updated state. (A `Promise.all` doesn't give you that; a transaction does.)

6.1 ACID: what a transaction protects (interview classic)

Letter	Means	ProjectHub example
Atomicity	all or nothing	tasks and project are deleted together, or neither
Consistency	valid state before and after; constraints hold at the end	no task points to a deleted project
Isolation	other sessions don't see your half-done work	another request never sees "tasks gone, project still there"; it waits on the lock
Durability	once COMMIT returns, it survives a crash	written to the transaction log on disk

Checkpoint answer, part 2

What does a transaction protect? A **multi-step change**. **Atomicity:** all steps happen or none (no half-deleted project). **Isolation:** nobody sees or overwrites the half-done state; other sessions wait for COMMIT or ROLLBACK.

Your answer (CHECK 7) explained atomicity with exactly our example: *"1st tasks are deleted and at that moment some problem can occur with the connection, so the second part is never done, and we end up with a project without tasks instead of a fully removed project. It guarantees no half-jobs."* Isolation was missing, and the re-check (b) added it: the other request waits until the transaction is committed or rolled back.

6.2 Autocommit: you've used transactions all along

Without `BEGIN TRANSACTION`, SQL Server runs **each statement as its own transaction** (autocommit). That's why in Topic 8 a failed INSERT left nothing behind ("The statement has been terminated"). You need `BEGIN TRANSACTION` only when **several** statements must stay together.

6.3 The SQL Server trap: an error does NOT undo the transaction

By default (`XACT_ABORT OFF`), an error like 547 cancels **only the failing statement**. The transaction stays open, the earlier statements are still there, and a COMMIT would save half the work. T2 proved it:

T2 real output: after the error the transaction is still committable (1) and the half-created project exists

```
ErrorNumber XactState ProjectStillThere
----------- --------- -----------------
547         1         1
```

Function	Values
`@@TRANCOUNT`	number of open transactions in this session (0 = none)

Function	Values
`XACT_STATE()`	**1** = open and committable; **-1** = open but doomed (only ROLLBACK allowed); **0** = no transaction

6.4 The correct pattern

```
SET XACT_ABORT ON;                -- any error dooms the WHOLE transaction (XACT_STATE() = -1)
BEGIN TRY
    BEGIN TRANSACTION;
    -- ... several statements ...
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;                        -- re-raise, like C# "throw;" (Topic 3)
END CATCH;
```

T3 real output: error **547**, `XACT_STATE() = -1`, rolled back. The final check found **0** demo projects: nothing half-done was left behind.

Your answer was half

You answered CHECK 8 with "no, we need to rollback in catch". "No" is right, but the pattern has **four** parts: `XACT_ABORT ON` + TRY/CATCH + ROLLBACK if open + **THROW**. Without THROW the error is swallowed, and the caller (our API) thinks the operation **succeeded**. That's the same reason `catch { }` without `throw;` is a bug in C#.

6.5 Transactions in C# (Topic 10)

- `SaveChangesAsync()` sends **all pending changes** (inserts, updates, deletes) **in one transaction automatically**. All saved, or nothing saved and an exception.
- Each `SaveChanges` call is **its own** transaction. Several calls as one unit: `await using var tx = await db.Database.BeginTransactionAsync(); ... await tx.CommitAsync();`. If you never commit, disposing `tx` rolls back.
- Your answer "committing them" (CHECK 10) was too vague. The re-check (c) got it right: 1 project + 3 tasks with an invalid third task, so **nothing saved, exception**.

Part 7. Locks and isolation: the real experiment

While a transaction changes a row, it holds a **lock** on it until COMMIT or ROLLBACK. SQL Server's default isolation level is **READ COMMITTED**: you never read uncommitted data, so a reader of that row **waits**. Our database has `READ_COMMITTED_SNAPSHOT` **off** (the on-premises default; Azure SQL has it on, and then readers see the last committed version instead of waiting).

Experiment in the cloud with two sqlcmd sessions. Session A changes project 1 and holds the transaction open for 10 seconds:

Session A (77)

```
USE ProjectHubDb;
SELECT is_read_committed_snapshot_on FROM sys.databases WHERE name = N'ProjectHubDb';
BEGIN TRANSACTION;
UPDATE dbo.Projects SET Name = N'Website redesign (editing)' WHERE Id = 1;
SELECT @@SPID AS SessionA;
WAITFOR DELAY '00:00:10';
ROLLBACK TRANSACTION;
GO
```

Session B (78), started 2 s later, with a 3 s lock timeout

```
USE ProjectHubDb;
SELECT @@SPID AS SessionB;
SELECT session_id, blocking_session_id, wait_type, command FROM sys.dm_exec_requests WHERE blocking_session_id <> 0;
GO
SET LOCK_TIMEOUT 3000;
SELECT Id, Name FROM dbo.Projects WHERE Id = 1;
GO
SELECT Id, Name FROM dbo.Projects WHERE Id = 2;
GO
```

Session B's output: project 1 times out (Msg 1222); project 2 answers at once (only the ROW is locked)

```
Changed database context to 'ProjectHubDb'.
SessionB
--------
78

(1 rows affected)
session_id blocking_session_id wait_type command
---------- ------------------- --------- -------

(0 rows affected)
Msg 1222, Level 16, State 51, Server 7a3b197acd25, Line 2
Lock request time out period exceeded.
Id Name
-- ----
2 Mobile app

(1 rows affected)
```

A third session, 1 s after B started: who is blocked by whom

```
session_id blocking_session_id wait_type wait_ms text
---------- ------------------- --------- ------- ----
78 77 LCK_M_S 988 (@1 tinyint)SELECT [Id],[Name] FROM [dbo].[Projects] WHERE [Id]=@1
```

- `blocking_session_id = 77`: session 78 waits for session 77. `LCK_M_S` = waiting for a **S**hared (read) lock.

- The waiting SQL shows `(@1 tinyint)SELECT ... WHERE [Id]=@1`: the server **auto-parameterized** our simple query (Topic 8's parameters, done by the server itself).
- **This is how you investigate "the API hangs"** in production: `sys.dm_exec_requests` + `blocking_session_id`.

Rules that follow

Keep transactions short. Never wait for a user, an HTTP call or a file inside one.
Never leave one open. A forgotten `BEGIN TRAN` in DBeaver blocks your own API. Check with `SELECT @@TRANCOUNT;` before closing an editor.

Part 8. Our scripts and their real output

Pinned to the commit where Topic 9's SQL was finished. 06 runs from the terminal (`./scripts/run-sql.sh sql/06-add-due-date.sql`); 07–09 run in DBeaver one query at a time; in 10, T1 runs one statement at a time and T2/T3 are selected as whole blocks. All five ran twice in the cloud with identical results (except `DaysOverdue`, which grows every day).

8.1 06-add-due-date.sql: a schema change on a table with data

```
IF COL_LENGTH(N'dbo.Tasks', N'DueDate') IS NULL
    ALTER TABLE dbo.Tasks ADD DueDate DATETIME2(7) NULL;
GO

-- Due dates chosen so the reports have something to show (today is mid-October 2026):
--   past + not Done   -> overdue (tasks 2, 5, 7)
--   past + Done       -> NOT overdue (tasks 1, 4): finished work is never late
--   future            -> not overdue yet (task 3)
--   NULL              -> no deadline (task 6): the NULL trap again
UPDATE dbo.Tasks SET DueDate = '2026-09-10T17:00:00' WHERE Id = 1;   -- Design mockups      (Done)
UPDATE dbo.Tasks SET DueDate = '2026-10-01T17:00:00' WHERE Id = 2;   -- Build landing page  (InProgress)
UPDATE dbo.Tasks SET DueDate = '2027-01-15T17:00:00' WHERE Id = 3;   -- SEO audit           (Todo)
UPDATE dbo.Tasks SET DueDate = '2026-09-05T17:00:00' WHERE Id = 4;   -- Write copy          (Done)
UPDATE dbo.Tasks SET DueDate = '2026-09-30T17:00:00' WHERE Id = 5;   -- Login screen        (InProgress)
UPDATE dbo.Tasks SET DueDate = NULL                  WHERE Id = 6;   -- Push notifications  (Todo)
UPDATE dbo.Tasks SET DueDate = '2026-10-10T17:00:00' WHERE Id = 7;   -- App store listing   (Todo)
GO

-- Check: the new column, filled.
SELECT Id, Title, Status, DueDate FROM dbo.Tasks ORDER BY Id;
```

- `ALTER TABLE ... ADD` changes a table that **already has rows**: what an **EF Core migration** does (Topic 10).
- The column is **nullable**, so existing rows simply get NULL. A `NOT NULL` column would need a `DEFAULT` for the existing rows, or the ALTER fails.
- `COL_LENGTH(...) IS NULL` = "the column doesn't exist yet": the script is **re-runnable** (like `IF DB_ID(...) IS NULL` in 01).
- The due dates are chosen to cover every case: overdue (2, 5, 7), past but Done (1, 4), future (3), no due date (6).
- `DueDate` exists only in SQL for now. In Topic 10 it's added to the C# `TaskItem` entity, and the EF migration creates the column from there.

Output of 06 (the check SELECT)

```
Id Title Status DueDate
-- ----- ------ -------
1 Design mockups 2 2026-09-10 17:00:00.0000000
2 Build landing page 1 2026-10-01 17:00:00.0000000
3 SEO audit 0 2027-01-15 17:00:00.0000000
4 Write copy 2 2026-09-05 17:00:00.0000000
5 Login screen 1 2026-09-30 17:00:00.0000000
6 Push notifications 0 NULL
7 App store listing 0 2026-10-10 17:00:00.0000000

(7 rows affected)
```

8.2 07-joins.sql

```
SELECT t.Id, t.Title, p.Name AS ProjectName
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId
ORDER BY p.Name, t.Id;

-- J2. LEFT JOIN: every PROJECT, with its tasks if it has any.
--     "Internal tools" has no tasks: it still appears, once, with NULL in every task column.
--     (INNER JOIN would drop it, exactly like J1 has no row for it.)
SELECT p.Id AS ProjectId, p.Name, t.Id AS TaskId, t.Title
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
ORDER BY p.Id, t.Id;

-- J3. THE LEFT JOIN TRAP: "every project, with its OPEN tasks".
--     J3a filters in WHERE: WHERE runs AFTER the join, so the NULL row of "Internal tools" fails
--     "t.Status <> 2" (NULL <> 2 is UNKNOWN) and the project disappears. The LEFT JOIN became an INNER JOIN.
SELECT p.Name, t.Title, t.Status
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
WHERE t.Status <> 2
ORDER BY p.Name, t.Title;

--     J3b puts the condition in ON: it decides WHICH TASKS match, and every project is still kept.
SELECT p.Name, t.Title, t.Status
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id AND t.Status <> 2
ORDER BY p.Name, t.Title;

-- J4. Topic 8's promise: Todo tasks of "Mobile app" by NAME, not by the magic number 2.
SELECT t.Id, t.Title, t.CreatedAt
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId
WHERE p.Name = N'Mobile app' AND t.Status = 0
ORDER BY t.CreatedAt;

-- REPORT 1: OVERDUE TASKS. Past the due date, not Done, with the project name, most overdue first.
--   * SYSUTCDATETIME() = "now" in UTC (C#: DateTime.UtcNow). Our dates are UTC too (Topic 1).
--   * DATEDIFF(DAY, a, b) = whole days from a to b.
--   * Task 6 has no due date (NULL): "NULL < now" is UNKNOWN, so it is NOT reported. That's the behavior we want here.
SELECT p.Name AS ProjectName,
       t.Id,
       t.Title,
       t.Status,
       t.DueDate,
       DATEDIFF(DAY, t.DueDate, SYSUTCDATETIME()) AS DaysOverdue
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId
WHERE t.DueDate < SYSUTCDATETIME()
  AND t.Status <> 2
ORDER BY t.DueDate;
```

Output of 07 (run on 17 Oct 2026)

```
Changed database context to 'ProjectHubDb'.
Id Title ProjectName
-- ----- -----------
5 Login screen Mobile app
6 Push notifications Mobile app
7 App store listing Mobile app
1 Design mockups Website redesign
2 Build landing page Website redesign
3 SEO audit Website redesign
4 Write copy Website redesign

(7 rows affected)
ProjectId Name TaskId Title
--------- ---- ------ -----
1 Website redesign 1 Design mockups
1 Website redesign 2 Build landing page
1 Website redesign 3 SEO audit
1 Website redesign 4 Write copy
2 Mobile app 5 Login screen
2 Mobile app 6 Push notifications
2 Mobile app 7 App store listing
3 Internal tools NULL NULL

(8 rows affected)
Name Title Status
---- ----- ------
Mobile app App store listing 0
Mobile app Login screen 1
Mobile app Push notifications 0
Website redesign Build landing page 1
Website redesign SEO audit 0

(5 rows affected)
Name Title Status
---- ----- ------
Internal tools NULL NULL
Mobile app App store listing 0
Mobile app Login screen 1
Mobile app Push notifications 0
Website redesign Build landing page 1
Website redesign SEO audit 0

(6 rows affected)
Id Title CreatedAt
-- ----- ---------
6 Push notifications 2026-09-07 09:00:00.0000000
7 App store listing 2026-09-08 09:00:00.0000000

(2 rows affected)
ProjectName Id Title Status DueDate DaysOverdue
----------- -- ----- ------ ------- -----------
Mobile app 5 Login screen 1 2026-09-30 17:00:00.0000000 17
Website redesign 2 Build landing page 1 2026-10-01 17:00:00.0000000 16
Mobile app 7 App store listing 0 2026-10-10 17:00:00.0000000 7
```

(continued)

```
(3 rows affected)
```

Report 1, read: three overdue tasks, most overdue first. Not in the report: tasks 1 and 4 (past due but **Done**), task 3 (due in 2027), task 6 (**no due date**: `NULL < now` is UNKNOWN, which is the behavior we want here).

8.3 08-group-by.sql

```
SELECT COUNT(*)       AS AllTasks,
       COUNT(DueDate) AS TasksWithDueDate,
       MIN(CreatedAt) AS FirstCreated,
       MAX(CreatedAt) AS LastCreated
FROM dbo.Tasks;

-- G2. GROUP BY one column: how many tasks in each status.
--     RULE: every column in SELECT is either in GROUP BY or inside an aggregate (COUNT, SUM, MIN, MAX, AVG).
--     LINQ: tasks.GroupBy(t => t.Status).Select(g => new { Status = g.Key, Tasks = g.Count() })
SELECT Status, COUNT(*) AS Tasks
FROM dbo.Tasks
GROUP BY Status
ORDER BY Status;

-- G3. THE COUNT(*) TRAP with a LEFT JOIN: tasks per project.
--     "Internal tools" has no tasks, but its LEFT JOIN row (with NULLs) is still a ROW, so COUNT(*) says 1.
SELECT p.Name, COUNT(*) AS Tasks_WRONG, COUNT(t.Id) AS Tasks_RIGHT
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
GROUP BY p.Id, p.Name
ORDER BY p.Name;

-- G4. HAVING = WHERE for GROUPS. Projects with MORE THAN 2 tasks that aren't Done.
--     WHERE filters ROWS before grouping (drop Done tasks), HAVING filters GROUPS after counting.
--     LINQ: tasks.Where(t => t.Status != Done).GroupBy(t => t.ProjectId).Where(g => g.Count() > 2)
--     (WHERE COUNT(*) > 2 is an ERROR: when WHERE runs, nothing has been counted yet.)
SELECT p.Name, COUNT(*) AS OpenTasks
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId
WHERE t.Status <> 2
GROUP BY p.Id, p.Name
HAVING COUNT(*) > 2;

-- REPORT 2: TASKS PER PROJECT. One row per project (including projects with NO tasks), with a count per status.
--   * LEFT JOIN keeps "Internal tools"; COUNT(t.Id) gives it 0 (G3).
--   * CASE WHEN ... THEN 1 ELSE 0 END turns "is this task Todo?" into 1 or 0; SUM adds them up.
--     This is "conditional aggregation": several counts in ONE pass over the rows.
--     For "Internal tools" t.Status is NULL, so every CASE gives 0 (the ELSE).
--   * Overdue uses the same rule as REPORT 1 (past due, not Done).
--   * The API version comes in Topic 11 (EF Core GroupBy, or a projection with t.Count(...)).
SELECT p.Id,
       p.Name,
       COUNT(t.Id)                                          AS TotalTasks,
       SUM(CASE WHEN t.Status = 0 THEN 1 ELSE 0 END)        AS Todo,
       SUM(CASE WHEN t.Status = 1 THEN 1 ELSE 0 END)        AS InProgress,
       SUM(CASE WHEN t.Status = 2 THEN 1 ELSE 0 END)        AS Done,
       SUM(CASE WHEN t.DueDate < SYSUTCDATETIME() AND t.Status <> 2 THEN 1 ELSE 0 END) AS Overdue
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
GROUP BY p.Id, p.Name
ORDER BY p.Name;
```

Output of 08

```
Changed database context to 'ProjectHubDb'.
AllTasks TasksWithDueDate FirstCreated LastCreated
-------- --------------- ------------ -----------
7 6 2026-09-02 09:00:00.0000000 2026-09-08 09:00:00.0000000

(1 rows affected)
Status Tasks
------ -----
0 3
1 2
2 2

(3 rows affected)
Name Tasks_WRONG Tasks_RIGHT
---- ----------- -----------
Internal tools 1 0
Mobile app 3 3
Website redesign 4 4

(3 rows affected)
Name OpenTasks
---- ---------
Mobile app 3

(1 rows affected)
Id Name TotalTasks Todo InProgress Done Overdue
-- ---- ---------- ---- ---------- ---- -------
3 Internal tools 0 0 0 0 0
2 Mobile app 3 2 1 0 2
1 Website redesign 4 1 1 2 1

(3 rows affected)
```

8.4 09-subqueries-exists.sql

```
SELECT Id, Title, CreatedAt
FROM dbo.Tasks
WHERE CreatedAt = (SELECT MAX(CreatedAt) FROM dbo.Tasks);

-- S2. IN (subquery): the inner query returns a LIST. Projects that have at least one Done task.
--     LINQ: var ids = tasks.Where(t => t.Status == Done).Select(t => t.ProjectId); projects.Where(p => ids.Contains(p.Id))
SELECT Id, Name
FROM dbo.Projects
WHERE Id IN (SELECT ProjectId FROM dbo.Tasks WHERE Status = 2)
ORDER BY Id;

-- S3. EXISTS: "is there AT LEAST ONE matching row?" -> true/false. Projects that have open (not Done) tasks.
--     The inner query mentions p.Id from the OUTER query: a CORRELATED subquery, checked per project.
--     SELECT 1 is a convention: EXISTS only cares whether a row exists, not what's in it.
--     The database can stop at the FIRST match (it doesn't count them all).
--     LINQ: projects.Where(p => tasks.Any(t => t.ProjectId == p.Id && t.Status != Done))  -> EF generates EXISTS
SELECT p.Id, p.Name
FROM dbo.Projects AS p
WHERE EXISTS (SELECT 1 FROM dbo.Tasks AS t WHERE t.ProjectId = p.Id AND t.Status <> 2)
ORDER BY p.Id;

-- S4. NOT EXISTS: projects WITHOUT any task. ("Internal tools")
--     LINQ: projects.Where(p => !tasks.Any(t => t.ProjectId == p.Id))
SELECT p.Id, p.Name
FROM dbo.Projects AS p
WHERE NOT EXISTS (SELECT 1 FROM dbo.Tasks AS t WHERE t.ProjectId = p.Id);

--     The same question with LEFT JOIN + IS NULL (also common; same result):
SELECT p.Id, p.Name
FROM dbo.Projects AS p
LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id
WHERE t.Id IS NULL;

-- S5. THE NOT IN TRAP: a NULL in the list makes NOT IN return NOTHING.
--     Question: "tasks whose due date is NOT one of the due dates of project 2's tasks".
--     Project 2's due dates are (2026-09-30, NULL, 2026-10-10). Task 6's NULL is in the list.
--     "x NOT IN (a, NULL)" means "x <> a AND x <> NULL", and "x <> NULL" is UNKNOWN -> never TRUE -> 0 rows.
SELECT Id, Title, DueDate
FROM dbo.Tasks
WHERE DueDate NOT IN (SELECT DueDate FROM dbo.Tasks WHERE ProjectId = 2);

--     NOT EXISTS has no such trap: it only asks "is there a matching row?". 4 rows (tasks 1-4, project 1).
--     (Task 6 itself isn't listed: its own DueDate is NULL, and NULL = anything is never TRUE.)
SELECT t.Id, t.Title, t.DueDate
FROM dbo.Tasks AS t
WHERE t.DueDate IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM dbo.Tasks AS t2 WHERE t2.ProjectId = 2 AND t2.DueDate = t.DueDate)
ORDER BY t.Id;
```

Output of 09: S1, S2, S3, S4 (twice), S5 (0 rows: the trap), S5b

```
Changed database context to 'ProjectHubDb'.
Id Title CreatedAt
-- ----- ---------
7 App store listing 2026-09-08 09:00:00.0000000

(1 rows affected)
Id Name
-- ----
1 Website redesign

(1 rows affected)
Id Name
-- ----
1 Website redesign
2 Mobile app

(2 rows affected)
Id Name
-- ----
3 Internal tools

(1 rows affected)
Id Name
-- ----
3 Internal tools

(1 rows affected)
Id Title DueDate
-- ----- -------

(0 rows affected)
Id Title DueDate
-- ----- -------
1 Design mockups 2026-09-10 17:00:00.0000000
2 Build landing page 2026-10-01 17:00:00.0000000
3 SEO audit 2027-01-15 17:00:00.0000000
4 Write copy 2026-09-05 17:00:00.0000000

(4 rows affected)
```

8.5 10-transactions.sql

```
BEGIN TRANSACTION;

SELECT @@TRANCOUNT AS OpenTransactions;                      -- 1: we are inside a transaction

DELETE FROM dbo.Tasks    WHERE ProjectId = 1;                -- (4 rows affected)
DELETE FROM dbo.Projects WHERE Id = 1;                       -- (1 row affected)

SELECT Id, Name FROM dbo.Projects ORDER BY Id;               -- inside the transaction: project 1 is gone

ROLLBACK TRANSACTION;                                        -- undo BOTH deletes (COMMIT would make them permanent)

SELECT Id, Name FROM dbo.Projects ORDER BY Id;               -- project 1 is back, with its 4 tasks
SELECT COUNT(*) AS TasksOfProject1 FROM dbo.Tasks WHERE ProjectId = 1;
SELECT @@TRANCOUNT AS OpenTransactions;                      -- 0: nothing left open
GO

-- T2. THE TRAP: an error does NOT automatically undo the transaction in SQL Server.
--     By default an error like 547 (CHECK) cancels only THAT STATEMENT. The transaction stays open,
--     and the statements before it are still there, waiting for COMMIT.
--     Create a project and its first task, and the task is invalid (Status 7).
--     TRY/CATCH (like C# try/catch) catches the error so we can look at the state:
--       XACT_STATE() =  1 -> a transaction is open and can still be COMMITTED (dangerous: half the work)
--       XACT_STATE() = -1 -> a transaction is open but doomed: only ROLLBACK is allowed
--       XACT_STATE() =  0 -> no transaction
SET XACT_ABORT OFF;                                          -- the default, written out for the demo
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO dbo.Projects (Name) VALUES (N'Half-created project');
    INSERT INTO dbo.Tasks (ProjectId, Title, Status) VALUES (SCOPE_IDENTITY(), N'Bad task', 7);   -- error 547
    COMMIT TRANSACTION;                                      -- never reached
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber,
           XACT_STATE()   AS XactState,                      -- 1: still committable!
           (SELECT COUNT(*) FROM dbo.Projects WHERE Name = N'Half-created project') AS ProjectStillThere;   -- 1
    -- A COMMIT here would save the project WITHOUT its task: half the work. We undo it:
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
END CATCH;
GO

-- T3. THE CORRECT PATTERN: XACT_ABORT ON + TRY/CATCH + ROLLBACK + THROW.
--     SET XACT_ABORT ON: any runtime error dooms the WHOLE transaction (XACT_STATE() = -1), not just the statement.
--     In CATCH: roll back if still open, then THROW the error again (like C# "throw;") so the caller knows.
--     This is what EF Core's SaveChanges does for you: all changes in one transaction, all or nothing (Topic 10).
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO dbo.Projects (Name) VALUES (N'All-or-nothing project');
    INSERT INTO dbo.Tasks (ProjectId, Title, Status) VALUES (SCOPE_IDENTITY(), N'Bad task', 7);   -- error 547
    COMMIT TRANSACTION;                                      -- never reached
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber,
           XACT_STATE()   AS XactState,                      -- -1: doomed, only ROLLBACK possible
           ERROR_MESSAGE() AS ErrorMessage;
```

(continued)

```
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    -- In real code the next line is:  THROW;   (re-raise). Left out here so the script continues to the check.
END CATCH;
SET XACT_ABORT OFF;
GO

-- Check: neither demo project exists. Nothing half-done is left behind.
SELECT COUNT(*) AS DemoProjects
FROM dbo.Projects
WHERE Name IN (N'Half-created project', N'All-or-nothing project');
GO
```

Output of 10: T1 (1, deletes, 2 projects, rollback, 3 projects, 4 tasks, 0), T2 (547, 1, 1), T3 (547, -1), check 0

```
Changed database context to 'ProjectHubDb'.
OpenTransactions
----------------
1

(1 rows affected)

(4 rows affected)

(1 rows affected)
Id Name
-- ----
2 Mobile app
3 Internal tools

(2 rows affected)
Id Name
-- ----
1 Website redesign
2 Mobile app
3 Internal tools

(3 rows affected)
TasksOfProject1
---------------
4

(1 rows affected)
OpenTransactions
----------------
0

(1 rows affected)

(1 rows affected)

(0 rows affected)
ErrorNumber XactState ProjectStillThere
----------- --------- -----------------
547 1 1

(1 rows affected)

(1 rows affected)

(0 rows affected)
ErrorNumber XactState ErrorMessage
----------- --------- ------------
547 -1 The INSERT statement conflicted with the CHECK constraint "CK_Tasks_Status". The conflict occurred in database "ProjectHubDb", table "dbo.Ta

(1 rows affected)
DemoProjects
------------
0
```

(continued)

```
(1 rows affected)
```

Building notes

My mistake while building 10: the first version of T2 let the 547 error escape. `run-sql.sh` uses `sqlcmd -b` ("stop on the first error"), so the script stopped there and T3 never ran. Fix: T2 catches its own error too and shows `XACT_STATE() = 1`, which also makes the T2 vs T3 contrast (1 vs -1) visible. `-b` is right for setup scripts and wrong for scripts that fail on purpose.
Side effect you can spot: T2 and T3 used up identity values. A ROLLBACK doesn't give ids back (Topic 8's gaps).

Part 9. DEBUG tasks with answers

No debugger for SQL: you investigate with small queries, the error messages and the server's own views. You went straight to the CHECK, so here are the answers.

Task	Answer
1. LEFT JOIN trap: why does Internal tools vanish in J3a?	Its row has `t.Status = NULL`; WHERE runs after the join; `NULL <> 2` is UNKNOWN; WHERE keeps only TRUE. Put the condition in ON (J3b).
2. COUNT trap: why does COUNT(*) say 1 for a project without tasks?	The LEFT JOIN creates one row for it (with NULLs); `COUNT(*)` counts rows. `COUNT(t.Id)` skips the NULL: 0.
3. Blocking lab (two DBeaver editors)	Editor A: `BEGIN TRANSACTION; UPDATE ... WHERE Id = 1;` (no commit). Editor B: `SELECT ... WHERE Id = 1` hangs. The `sys.dm_exec_requests` query shows B blocked by A. A: `ROLLBACK` and B finishes with the **old** name. `WHERE Id = 2` in B is not blocked (row lock). Part 7 has the real run.
4. NOT IN trap: why does S5 return 0 rows?	Project 2's due dates include NULL; `x NOT IN (a, NULL, b)` contains `x <> NULL`, which is UNKNOWN for every x.
5. Your query: projects with their number of overdue tasks, only those with at least one	Below. Result: Mobile app 2, Website redesign 1.

DEBUG task 5 (model answer, verified)

```
SELECT p.Name, COUNT(*) AS OverdueTasks
FROM dbo.Tasks AS t
INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId
WHERE t.DueDate < SYSUTCDATETIME() AND t.Status <> 2     -- rows: overdue tasks only
GROUP BY p.Id, p.Name
HAVING COUNT(*) >= 1;                                     -- groups: at least one
```

Interview follow-up: the HAVING is **redundant** here. After an INNER JOIN and that WHERE, every group already has at least one row. It would matter with a LEFT JOIN (zero-task groups exist) or with a higher threshold (`> 1`).

Investigation toolbox (adds to Topic 8's)

Question	Tool
Is a transaction still open in my session?	`SELECT @@TRANCOUNT;` (0 = none)
Can this transaction still be committed?	`SELECT XACT_STATE();` (1 yes, -1 doomed, 0 none)
Why is my query / the API hanging?	`sys.dm_exec_requests` with `blocking_session_id <> 0` (+ `sys.dm_exec_sql_text` for the SQL)
Does a LEFT JOIN keep the rows I expect?	count rows with and without the WHERE; check for NULL-filled rows
Are my counts right?	compare `COUNT(*)` with `COUNT(t.Id)`

Part 10. Your answers, corrections, interview Q&A

10.1 CHECK: your answers

Question	Your answer	Verdict / complete answer
1. Why a JOIN?	"retrieve info from 2 tables in one go; without it the DB would scan one table, then the 2nd, then match"	**Half:** "one go" right; that scanning is what a JOIN does inside. Without it: N+1 queries in your code, or duplicated data. Re-check: **right**
2. INNER vs LEFT	"inner: only truthy rows; left: all rows"	**Half:** match on both sides vs all rows of the LEFT table + NULLs; example missing
3. LEFT JOIN + WHERE	"WHERE removes the row where status is null; condition belongs to the join"	**Right** (in ON)
4. COUNT(*) vs COUNT(t.Id)	"* all rows even unreal (falsy); t.Id valid truthy rows"	**Right idea, wrong words:** COUNT(col) skips only NULL; 0 and '' are counted
5. WHERE vs HAVING	"having is where but for groups"	**Right.** + WHERE runs before grouping, can't use aggregates
6. EXISTS, NOT IN	"checks if something exists in a list; Any(); NOT IN breaks with NULL"	**Right.** + stops at the first match; correlated
7. What a transaction protects	"no half-jobs" with the delete-project example	**Atomicity right.** Isolation missing; re-check (b): **right**
8. Does an error undo it?	"no, we need to rollback in catch"	**Half:** XACT_ABORT ON + TRY/CATCH + ROLLBACK + THROW
9. Why short?	"they lock, other sessions are waiting"	**Right.** + until COMMIT/ROLLBACK; requests hang or time out (1222)
10. SaveChanges	"committing them"	**Too vague:** all pending changes in ONE automatic transaction. Re-check (c): **right**

10.2 Re-check (round 2)

Question	Your answer	Verdict
a) Why a JOIN, and what without it?	"so we receive data matched from 2 tables in one query; without it WE would query one table and then query per row (tasks-project)"	**Right** (that's N+1)
b) Another request SELECTs project 1 mid-transaction	"it waits for the transaction to finish (committed or rolled back)"	**Right.** It waits on the lock; old row after ROLLBACK, "not found" after COMMIT; may time out
c) SaveChanges, 1 project + 3 tasks, 3rd invalid	"nothing saved, exception"	**Right**

10.3 Mistakes and corrections in this topic

- **You:** "without a JOIN the DB would scan both tables and match". Correction: that IS a join's work; without a JOIN the work moves into your code (N+1).

- **You:** "truthy / falsy" for JOIN and COUNT. Correction: SQL has TRUE / FALSE / UNKNOWN, and COUNT skips only NULL.
- **You:** "rollback in catch" as the whole pattern. Correction: XACT_ABORT ON + ROLLBACK if open + THROW.
- **Me:** T2 first let its error escape; `sqlcmd -b` stopped the script before T3. Fixed by catching it and showing `XACT_STATE()`.

10.4 Interview questions with model answers

Q: Why is a JOIN needed?

Data is normalized: each fact is stored once (the project name in Projects, only ProjectId in Tasks). A JOIN combines the rows in one query by matching FK to PK. Without it: N+1 queries from the application, or duplicated data that can disagree.

Q: INNER JOIN vs LEFT JOIN?

INNER keeps rows with a match on both sides. LEFT keeps every row of the left table, with NULLs where there's no match. Tasks with project names: INNER. A per-project report including projects with 0 tasks: LEFT.

Q: Why does a WHERE on the right table break a LEFT JOIN?

WHERE runs after the join; the unmatched rows have NULLs there, NULL comparisons are UNKNOWN, and WHERE drops them, so it becomes an INNER JOIN. Put the condition in ON.

Q: COUNT(*) vs COUNT(column)?

COUNT(*) counts rows; COUNT(column) counts non-NULL values. After a LEFT JOIN, COUNT(*) gives 1 for a parent without children; count a right-table column instead.

Q: WHERE vs HAVING?

WHERE filters rows before grouping and can't use aggregates; HAVING filters groups after aggregation.

Q: What's the logical order of a SELECT?

FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, OFFSET/FETCH. Hence no aggregates in WHERE and no SELECT aliases in WHERE.

Q: What does EXISTS do? Why prefer NOT EXISTS to NOT IN?

EXISTS is true if the subquery returns at least one row and can stop at the first match (LINQ Any()). NOT IN returns nothing if the list contains a NULL; NOT EXISTS doesn't have that problem.

Q: What does a transaction protect? Explain ACID.

A multi-step change. Atomicity: all or nothing. Consistency: constraints hold before and after. Isolation: others don't see the half-done state. Durability: committed data survives a crash.

Q: Does an error inside a transaction roll it back in SQL Server?

Not by default: only the failing statement is cancelled and the transaction stays open (XACT_STATE 1). Use SET XACT_ABORT ON with TRY/CATCH, ROLLBACK if @@TRANCOUNT > 0, and THROW.

Q: Why keep transactions short?

Locks are held until COMMIT/ROLLBACK; other sessions wait (READ COMMITTED) and requests hang or time out. Never wait for users or external calls inside a transaction.

Q: How do you find out why queries are hanging?

sys.dm_exec_requests where blocking_session_id <> 0 shows who waits on whom and the wait type (e.g. LCK_M_S); sys.dm_exec_sql_text shows the SQL.

Q: What does EF Core's SaveChanges do with transactions?

It sends all pending changes in one transaction automatically: all saved or none, plus an exception. Several SaveChanges as one unit: Database.BeginTransactionAsync() + CommitAsync().

Q: What is the N+1 problem?

Loading a list with one query and then each item's related data with one query per item: N+1 round trips instead of one JOIN. Common with ORM lazy loading (Topic 11).

Q: How do you add a column to a table that already has data?

ALTER TABLE ... ADD. Nullable: existing rows get NULL. NOT NULL: give a DEFAULT. In EF Core this is a migration.

Part 11. Summary

Key takeaways

JOIN: data is normalized (each fact once); a JOIN recombines it in one query, FK = PK. The alternative is N+1 queries.
INNER = matches on both sides; **LEFT** = all left rows + NULLs. Right-table conditions of a LEFT JOIN go in **ON**.
Order: FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, OFFSET/FETCH.
GROUP BY: every SELECT column grouped or aggregated; **HAVING** filters groups; **COUNT(t.Id)** after a LEFT JOIN; CASE + SUM for several counts.
EXISTS = Any(), stops at the first match; **NOT EXISTS** instead of NOT IN (NULL trap).
Transactions protect multi-step changes: atomicity (all or nothing) + isolation (no half-done state visible).
SQL Server: an error doesn't undo the transaction; use XACT_ABORT ON + TRY/CATCH + ROLLBACK + THROW.
Locks last until COMMIT/ROLLBACK: keep transactions short; find blockers with `sys.dm_exec_requests`.
EF Core: navigation properties become JOINs, `Any()` becomes EXISTS, `SaveChanges` = one transaction.

Cheat sheet

Need	SQL
rows of two tables	`FROM dbo.Tasks AS t INNER JOIN dbo.Projects AS p ON p.Id = t.ProjectId`
keep parents without children	`FROM dbo.Projects AS p LEFT JOIN dbo.Tasks AS t ON t.ProjectId = p.Id`
filter the right side of a LEFT JOIN	`... ON t.ProjectId = p.Id AND t.Status <> 2`
count per group	`SELECT p.Name, COUNT(t.Id) ... GROUP BY p.Id, p.Name`
filter groups	`HAVING COUNT(*) > 2`
count by condition	`SUM(CASE WHEN t.Status = 0 THEN 1 ELSE 0 END)`
"has any" / "has none"	`WHERE [NOT] EXISTS (SELECT 1 FROM dbo.Tasks AS t WHERE t.ProjectId = p.Id)`
now (UTC), days between	`SYSUTCDATETIME()`, `DATEDIFF(DAY, a, b)`
all or nothing	`SET XACT_ABORT ON; BEGIN TRY BEGIN TRAN ... COMMIT END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK; THROW; END CATCH`
open transactions?	`SELECT @@TRANCOUNT;` / `SELECT XACT_STATE();`
who blocks whom	`SELECT session_id, blocking_session_id, wait_type FROM sys.dm_exec_requests WHERE blocking_session_id <> 0;`
add a column	`ALTER TABLE dbo.Tasks ADD DueDate DATETIME2(7) NULL;`

Checklist: can you do this without notes?

- Explain why a JOIN is needed and what N+1 is.
- Write Report 1 (overdue tasks with project names) and say why tasks 1, 3, 4 and 6 are not in it.
- Write Report 2 (tasks per project, including projects with no tasks) and explain COUNT(t.Id).
- Explain the LEFT JOIN + WHERE trap and the NOT IN trap with the NULL rule.
- Explain ACID with "delete a project and its tasks".

- Write the XACT_ABORT + TRY/CATCH + THROW pattern and say what each line is for.
- Explain what another request sees while your transaction runs, and how to find who blocks it.
- Say what SaveChanges does with transactions.

What comes next: Topic 10 (EF Core basics)

Replace the in-memory `ProjectService` storage with SQL Server through **EF Core**: `DbContext`, `DbSet<T>`, the SQL Server provider, the connection string (the `Database=` part from Topic 8's master trap), and **migrations** that create the schema from the C# entities (including the new `DueDate`). Checkpoint: the schema comes from a migration, and CRUD persists across restarts.